

LangGraph를 활용한 Q&A 시스템

State, Node, Edge 컴포넌트를 활용한 복잡한 워크플로우 모델링



LangGraph 핵심 컴포넌트 개요



상태 (State)

- 워크플로우 전반에 걸쳐 공유되고 업데이트되는 중앙 데이터 저장소
- 파이썬의 TypedDict 또는 Pydantic 모델로 정의
- 질문-답변 시스템에서는 사용자의 질문, 선정된 답변 역할, 생성된 답변, 품질 검사 결과 등을 포함



노드 (Node)

- 워크플로우의 개별 작업 단위(Task)를 나타내는 파이썬 함수
- 현재 상태(State)를 입력으로 받아 특정 작업을 수행하고 업데이트된 상태를 반환
- 질문-답변 시스템에서는 질문 분류 및 답변 역할 선정, 답변 작성, 품질 검사 등 각 단계를 노드로 표현

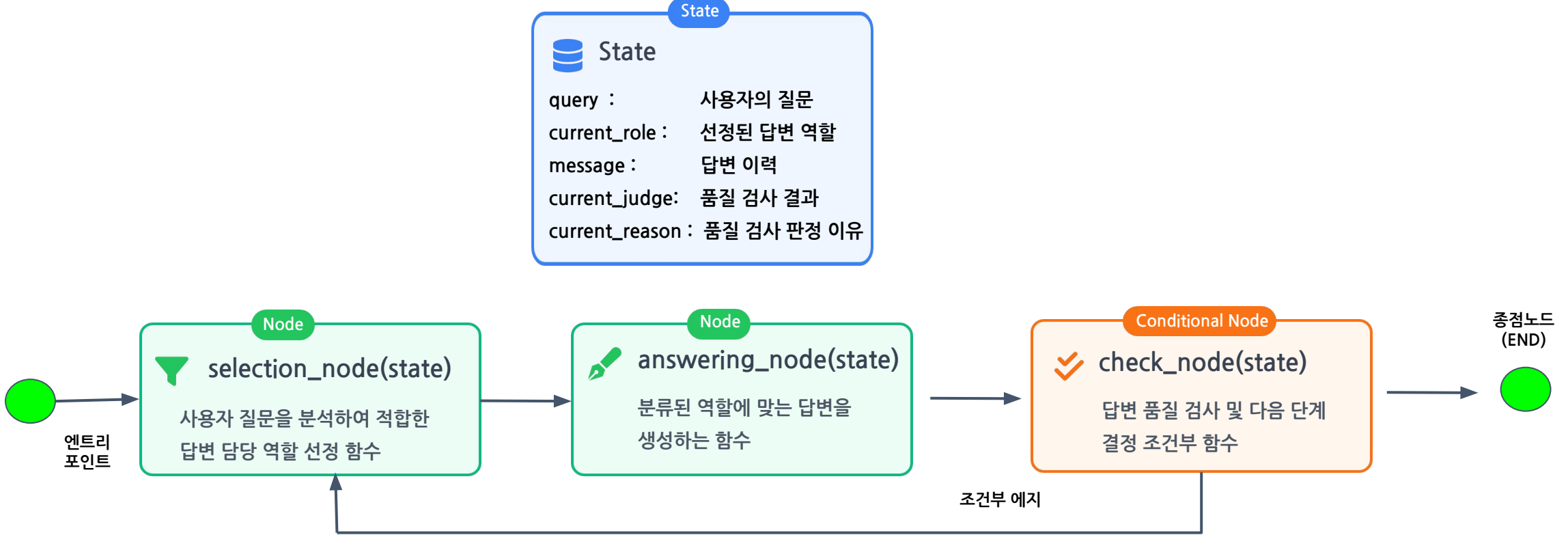


엣지 (Edge)

- 노드 간의 연결을 정의하며, 워크플로우의 흐름을 제어
- 일반 엣지: 특정 노드에서 다른 노드로 무조건적으로 흐름을 전달
- 조건부 엣지: 특정 조건에 따라 다음 실행할 노드를 동적으로 결정

질문-답변 시스템에서의 LangGraph 컴포넌트 상호작용

● 상태 (State) → ● 노드 (Node) → ● 엣지 (Edge)



LangGraph 질문-답변 시스템 구현

그래프의 상태 정의

```

class State(BaseModel):
    query: str = Field(..., description="사용자의 질문")
    current_role: str = Field(default="", description="선정된 답변 역할")
    messages: Annotated[list[str], operator.add] = Field(default=[], description="답변 이력")
    current_judge: bool = Field(default=False, description="품질 검사 결과")
    judgement_reason: str = Field(default="", description="품질 검사 판정 이유")
  
```

노드 및 엣지 연결

```

# 엔트리 포인트 : 그래프의 시작 노드를 지정하는 에지
workflow.set_entry_point("selection")
workflow.add_edge("selection", "answering")
workflow.add_edge("answering", "check")
# 조건부 에지
workflow.add_conditional_edges(
    "check",
    lambda state: state.current_judge,
    {True: END, False: "selection"}
)
  
```

역할 정의

```
ROLES = {  
  "1": {  
    "name": "일반 지식 전문가" ,  
    "description": "폭넓은 분야의 일반적인 질문에 답변" ,  
    "details": "폭넓은 분야의 일반적인 질문에 대해 정확하고 이해하기 쉬운 답변을 제공하세요."  
  } ,  
  "2": {  
    "name": "생성형 AI 제품 전문가" ,  
    "description": "생성형 AI 관련 제품, 기술에 관한 전문적인 질문에 답변" ,  
    "details": "생성형 AI 관련 제품, 기술에 관한 전문적인 질문에 대해 최신 정보와 깊은 통찰력을 제공하세요."  
  } ,  
  "3": {  
    "name": "카운슬러" ,  
    "description": "개인적인 고민이나 심리적인 문제에 대해 지원 제공" ,  
    "details": "개인적인 고민이나 심리적인 문제에 대해 공감하고 지원적인 답변을 제공하고, 가능하다면 적절한 조언도 해주세요."  
  }  
}
```

- 답변을 생성하기 전에 답변을 담당할 역할을 선택하는 단계로 미리 정의된 역할을 LLM에게 선택하도록 할 것입니다.
- 미리 정의할 수 있는 내용은 LLM에게 생성시키지 않는 것이 비용 측면이 응답 속도 측면에서 유리합니다.

상태(State) 설계

State 정의

```
class State(BaseModel):  
    # 필수 필드  
    query: str = Field(..., description="사용자의 질문")  
    current_role: str = Field(default="", description="선정된 답변 역할")  
    messages: Annotated[list[str], operator.add] = Field(default=[], description="답변 이력")  
    current_judge: bool = Field(default=False, description="품질 검사 결과")  
    judgement_reason: str = Field(default="", description="품질 검사 판정 이유")
```

상태(State)의 역할

- 워크플로우 전반에 걸쳐 공유되고 업데이트되는 중앙 데이터 저장소
- 각 노드가 작업을 수행하며 상태를 읽고 업데이트
- Q&A 시스템의 모든 데이터를 일관되게 관리

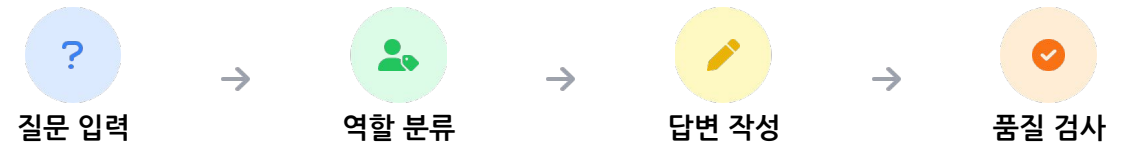
상태 흐름 시각화

State

중앙 데이터 저장소

query: str	사용자의 질문	→
current_role: str	선정된 답변 역할	→
judgement_reason: str	품질 검사 판정 이유	→
current_judge: bool	품질 검사 결과	→
messages: Annotated[list, operator.add]	답변 이력	↺

상태 업데이트 흐름



노드(Node) 구현

노드(Node)는 LangGraph 워크플로우의 개별 작업 단위를 나타내는 파이썬 함수로, 현재 상태를 입력으로 받아 특정 작업을 수행하고 업데이트된 상태를 반환합니다.



질문 분류 및 역할 선정

```
def selection_node(state: State) -> dict[str, Any]:
```

- 사용자 질문을 분석하여 적절한 답변 생성 담당 역할을 선정
- LLM 호출을 통해 질문 내용에 따라 일반 지식 전문가, 생성형 AI 제품 전문가, 카운슬러 중 하나를 선정
- 선정된 역할을 상태에 업데이트



답변 생성

```
def answering_node(state: State) -> dict[str, Any]:
```

- 사용자의 질문과 선정된 역할에 따라 질문에 대한 답변 생성
- 역할에 맞춘 전문 지식을 활용하여 답변 생성
- 생성된 답변을 상태에 업데이트



품질 검사

```
def check_node(state: State) -> dict[str, Any]:
```

- 작성된 답변의 품질을 검사하고 문제 여부를 판단
- LLM 호출을 통해 답변의 정확성, 충분성, 관련성 등을 평가
- 검사 결과(통과/실패)와 이유를 상태에 업데이트

엣지(Edge) 연결 및 흐름 제어

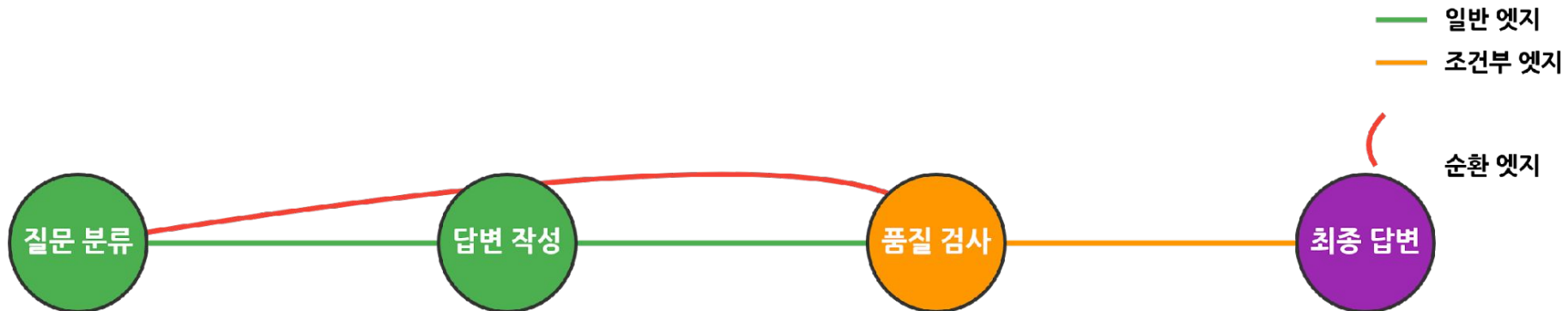
→ 일반 엣지 (Normal Edge)

- 특정 노드에서 다른 노드로 무조건적으로 흐름을 전달
- Q&A 시스템에서 필수적인 순차적 흐름 정의
- `add_edge("selection", "answering") :`
selection 노드에서 answering 노드로 에지를 연결

조건부 엣지 (Conditional Edge)

- 특정 조건에 따라 다음 실행할 노드를 동적으로 결정
- 품질 검사 결과에 따라 다른 경로로 분기
- 예: 품질 검사 통과 시 최종 답변 제시, 실패 시 질문 재분류

품질 검사 기반 순환 구조



그래프 구성 및 시각화

</> StateGraph를 통한 워크플로우 구축

```
# StateGraph 초기화
workflow = StateGraph (AgentState)

# 노드 추가
workflow . add_node ( "selection" , selection_node)
workflow . add_node ( "answering" , answering_node)
workflow . add_node ( "check" , check_node)

# 그래프의 시작 노드 설정 (엔트리 포인트)
workflow . set_entry_point ( "selection" )

# 엣지 연결
workflow . add_edge ( "selection" , "answering" )
workflow . add_edge ( "answering" , "check" )

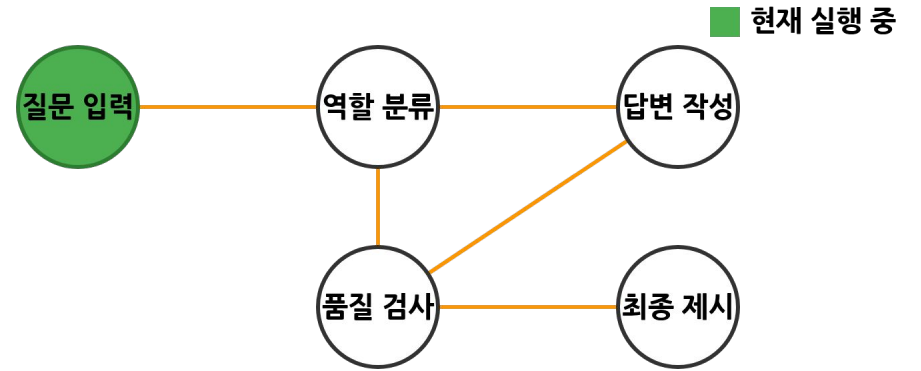
# 조건부 엣지 추가
workflow . add_conditional_edges (
    "check" ,
    lambda state: state.current_judge,
    {True: END, False: 'selection'}

# 그래프 컴파일
capp = workflow . compile ()

# 그래프 실행
initial_state = State(query="생성형 AI에 관해 알려주세요 ")
result = app.invoke(initial_state)

# 그래프 시각화
Image(app.get_graph().draw_png())
```

Mermaid를 활용한 그래프 시각화



주요 메서드:

- `get_graph()` : 워크플로우의 구조를 가져옵니다
- `draw_png()` : png 이미지로 시각화합니다.
- `compile()` : 실행 가능한 애플리케이션으로 컴파일합니다